



Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

MONGODB PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Databases

TOTAL: 10 QUESTIONS

Q1 What type of database is MongoDB and what are its **core** strengths? **CRAB Model**

Q2 How does MongoDB guarantee consistency and handle **distributed** transactions? **Architecture**

Q3 What index types does MongoDB support and when **do you** use each? **Indexing**

Q4 How do you design a schema or data model in MongoDB? **Designing**

Q5 How does MongoDB scale horizontally? **SSO / IdP**

Q6 How do you monitor and tune slow queries in MongoDB? **Availability**

Q7 What backup and restore strategies does MongoDB support? **Backup/Restore**

Q8 How do you handle schema migrations in MongoDB? **Compliance**

Q9 What security features does MongoDB provide? **Testing**

Q10 What are common anti-patterns when using MongoDB in **production**? **Tuning**



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 MongoDB is a relational, document, key-value, graph, or time-series store. Mitigation

MongoDB is a relational, document, key-value, graph, or time-series store. Its strengths — such as ACID guarantees, horizontal scale, or flexible schema — guide the workloads it is best suited for.

A6 MongoDB provides a slow query log, explain/explain plan, or profiling tools to surface inefficient queries. Observability

MongoDB provides a slow query log, explain/explain plan, or profiling tools to surface inefficient queries. Common fixes include adding indexes, rewriting queries, or increasing cache size.

A2 MongoDB provides either full ACID transactions or eventual consistency. Primitives

MongoDB provides either full ACID transactions or eventual consistency depending on its CAP theorem positioning. Transaction support varies from none (simple K/V stores) to serializable isolation (relational DBs).

A7 MongoDB supports logical backups (export SQL or BSON), physical backups (filesystem snapshot), and continuous replication to a standby. Automation

MongoDB supports logical backups (export SQL or BSON), physical backups (filesystem snapshot), and continuous replication to a standby. Point-in-time recovery requires WAL/oplog archiving on top of backups.

A3 MongoDB offers B-tree, hash, full-text, spatial, or composite indexes. Hardening

MongoDB offers B-tree, hash, full-text, spatial, or composite indexes depending on the engine. Choose based on query patterns: B-tree for range queries, hash for equality lookups, full-text for search.

A8 Schema migrations in MongoDB are managed with versioned migration files. Governance

Schema migrations in MongoDB are managed with versioned migration files applied in order by a migration tool. Migrations should be idempotent and tested in staging before production to avoid downtime.

A4 Schema design in MongoDB involves normalizing relations (relational DB) or denormalizing for access patterns (document/key-value). Optimization

Schema design in MongoDB involves normalizing relations (relational DB) or denormalizing for access patterns (document/key-value). Understanding query needs upfront prevents costly migrations later.

A9 MongoDB supports role-based access control, encrypted connections (TLS), encryption at rest, and audit logging. Assessment

MongoDB supports role-based access control, encrypted connections (TLS), encryption at rest, and audit logging. Always restrict user privileges to the minimum needed and disable anonymous access in production.

A5 MongoDB scales via replication (read replicas) for read-heavy workloads and sharding (partitioning data by key) for write-heavy ones. Federation

MongoDB scales via replication (read replicas) for read-heavy workloads and sharding (partitioning data by key) for write-heavy ones. Some MongoDB implementations handle this automatically; others require manual configuration.

A10 Common mistakes include selecting all columns instead of needed fields, missing indexes on frequently queried columns, running migrations without backups, and storing large blobs directly in the database instead of object storage. Optimization

Common mistakes include selecting all columns instead of needed fields, missing indexes on frequently queried columns, running migrations without backups, and storing large blobs directly in the database instead of object storage.